Università degli Studi di Verona

CORSO DI LAUREA IN INFORMATICA

Basi di dati

Federico Brutti
federico.brutti@studenti.univr.it

L'identificatore all'interno della relazione NON. CI. VA! - Belussi A.

Contents

1 Progettazione di una base di dati **3**
1.1 Applicazioni gestionali e DBMS 3
1.2 Creazione di una base di dati 4
1.3 Progettazione concettuale 5
1.4 Progettazione logica 11
1.5 Esercizi svolti 11

2 Interazione con un database **12**

3 RDBMS e SQL in postgreSQL **13**

4 Query e transazioni **14**

5 Accesso a database da programmi Python **15**

6 Uso di DBMS, NoSQL e MongoDB **16**

7 Database Management Systems **17**

8 Interrogazioni in SQL **18**

9 Architettura NoSQL MongoDB **19**

Chapter 1

Progettazione di una base di dati

1.1 Applicazioni gestionali e DBMS

Le **Applicazioni** informatiche in ambito gestionale sono nate negli anni '70 e sono caratterizzate da algoritmi di elaborazione semplici e una grande quantità di dati condivisi da più applicazioni, ambo derivanti dall'ambiente in cui sono introdotte, chiamato **sistema informativo**. Quest'ultimo è definito formalmente come l'*insieme delle attività umane e dei dispositivi di memorizzazione ed elaborazione atte alla gestione e all'organizzazione di informazioni di interesse per un'organizzazione.* Il sistema informativo è composto a sua volta da due componenti:

- **Dato**: Elemento di conoscenza di base, costituito da simboli da elaborare.
- **Informazione**: Interpretazione dei dati che consente di ottenere conoscenza di fatti e situazioni.

È possibile studiare il comportamento dei sistemi informativi mediante diagrammi di flusso, i quali **definiscono** archivi, sorgenti, utenti, procedure e flussi di dati, quindi rappresentano le informazioni come collezioni di dati. Questi elementi portano a quella che è chiamata **soluzione convenzionale**, che mostra i vari programmi interagire col file system, il quale mantiene i dati. Tuttavia presenta alcuni problemi, come per esempio un accesso lento ai dati e la loro ridondanza.

Un'altra soluzione pone come middleware posto tra le applicazioni e il file system un **Database Management System** (DBMS), un sistema che gestisce le **basi di dati** su memoria secondaria, assicurando affidabilità, privatezza e accesso efficiente. Presenta inoltre due vantaggi principali:

- **Maggiore astrazione**: I dati sono visti come oggetti o righe di tabelle quando il DBMS si interfaccia con le applicazioni.
- **Maggiore efficacia**: L'accesso ai dati è basato su un linguaggio di interazione.

In particolare, i **linguaggi di interazione** si dividono in due categorie: la prima è atta alla **definizione dei dati** (DDL), mentre la seconda concerne la **manipolazione dei dati** (DML), a sua volta suddivisa in **linguaggi di interrogazione**, per estrarre informazioni dal DB, e **di manipolazione**, per popolare il DB o modificarne i dati. Il DBMS fornisce inoltre un **Modello dei Dati**, l'insieme dei costrutti per la descrizione di struttura e proprietà dell'informazione all'interno del DB. Ad oggi, vengono utilizzati i seguenti modelli:

- Modello relazionale: SQL92 o SQL2
- Modello a oggetti
- Modello object-relational: SQL99 o SQL3
- Modello basato su documenti: JSON
- Modelli NoSQL: Basati su colonne, costrutti chiave-valore

Bisogna tuttavia fare attenzione a distinguere il modello dei dati dallo **schema del DB**, la descrizione di struttura e proprietà della base di dati che fa uso dei costrutti del modello, e dall'**istanza del DB**, costituita dagli effettivi valori che popolano le strutture dati in una determinata istanza. In soldoni, il modello definisce i costrutti, lo schema la loro relativa implementazione e l'istanza una situazione specifica con determinati valori.

Passiamo ora all'architettura generale dei DBMS: presentano tre livelli: **esterno**, che descrive una porzione dello schema logico per un'interfaccia specifica, **logico**, la rappresentazione di struttura e proprietà del DB, definite con il modello dei dati, e **interno**, rappresentante il DB mediante le strutture fisiche di memorizzazione. In un contesto tale è necessario chiarire che le basi di dati devono mantenere due tipi di indipendenza:

- **Fisica**: Schema logico è indipendente da quello interno, facendo sì che la variazioni sul secondo non impattino il primo.
- **Logica**: Schemi esterni indipendenti da quello logico, facendo sì che le variazioni nel secondo, purché non rimuovano dati, non impattano i primi.

1.2 Creazione di una base di dati

Per progettare adeguatamente una base di dati è necessario studiare il ciclo di vita del processo di automazione all'interno di un sistema informativo. In primo luogo è richiesto uno **studio di fattibilità** per definire costi ed eventuali alternative; segue la **raccolta e analisi dei requisiti** del sistema, producendo una descrizione completa ma informale, per poi terminare con l'effettiva **progettazione**. Quest'ultima fase è suddivisa in due attività:

- **Progettazione dei dati**: Descrizione formale dei dati attraverso lo schema, comprende l'implementazione del DBMS seguita dalla validazione e collaudo.
- **Progettazione delle applicazioni**: Descrizione formale delle applicazioni attraverso le specifiche.

In questo corso ci concentreremo sulle metodologie di progettazione dei dati. Un buon metodo è costituito da una decomposizione iniziale in passi dell'attività di progetto, attuazione di strategie e modelli di riferimento da seguire, come anche di criteri di scelta da applicare. La metodologia deve inoltre essere generale, semplice da usare, ma soprattutto in grado di produrre un risultato completo e corretto.

Nel concreto, la progettazione dei dati presenta tre fasi principali: **concettuale**, per rappresentare il contenuto informativo del DB formalmente e indipendente da tecnologia e operazioni, portando alla creazione dello schema concettuale, **logica** per tradurre lo schema concettuale in quello logico, aderente al modello dei dati del DBMS scelto, e **fisica**, per completare lo schema logico coi parametri relativi alla memorizzazione fisica dei dati e metodi di accesso per garantire efficienza, fornendo lo schema fisico.

1.3 Progettazione concettuale

Il modello storicamente usato per la progettazione concettuale delle basi di dati è quello **Entità-Relazione** (ER); propone un insieme di strumenti grafici formali e indipendenti dalla tecnologia[1], detti **costrutti**, per specificare struttura e proprietà dei dati da rappresentare al suo interno.

La definizione di ogni costrutto si ha specificando il loro significato, sintassi grafica e relative istanze; quelli di base sono:

- **Entità**: Classe di oggetti con attributi comuni, esistenza autonoma e identificazione univoca, rappresentate con un rettangolo contenente il loro nome.
- **Relazione**: Legame logico tra due o più entità, rappresentate da un rombo a cui si collegano attraverso linee le entità coinvolte. Il nome è posto vicino alla forma geometrica.
- **Attributo**: Proprietà elementare di un'entità o una relazione; associa a ogni loro istanza un singolo valore appartenente a un dominio, indicandolo con una piccola riga alla cui estremità è posto un pallino vuoto con a fianco il suo nome.

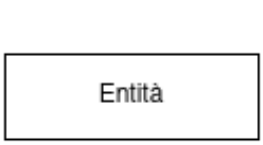
Ognuno di questi costrutti si può **istanziare**, ovvero concretizzare, all'interno della base di dati; sono annotati con $I(nomeCostrutto)$ e i relativi insiemi sono detti **popolazioni**.

[1] Privi di qualsivoglia ottimizzazioni o semplificazioni che possono avere luogo in base agli strumenti scelti.

Alla nascita della base di dati ogni popolazione è vuota, poiché nessun costrutto è stato ancora istanziato. Istanziare un'entità significa aggiungere un oggetto alla base di dati e farlo per un attributo corrisponde ad associare un valore del dominio che assume a un'istanza di entità o relazione. Per queste ultime in particolare, definiamo le loro istanze come n-uple di istanze di entità; formalmente:

$$(e_1, ..., e_n), \text{ con } e_i \in I(E_i), 1 \leq i \leq n$$

E la relativa popolazione rappresenta l'insieme delle n-uple delle due entità coinvolte relazionate in un certo istante di tempo. Si tratta di una rappresentazione univoca, quindi non è possibile annotare più n-uple allo stesso modo.



(a) Entità



(b) Relazione



(c) Attributo

Durante la progettazione è comune avere la necessità di verificare la correttezza delle entità e quindi di distinguere quelle problematiche; il costrutto usato è l'**identificatore di entità**, il quale, data un'entità E, si compone di un'insieme di suoi attributi e/o relazioni capaci di identificare univocamente le sue istanze. Ciò è possibile, naturalmente, solo se non è consentita l'esistenza di più istanze di entità che presentino proprietà uguali. Graficamente si rappresenta in più modi:

- Se è composto da un solo attributo, si colora l'intero pallino.
- Se è composto da più attributi, si disegna una riga perpendicolare che attraversa entrambi i loro gambi, la quale avrà un pallino colorato in una delle sue estremità[2]. Si dicono **identificatori interni**.
- Se è composto anche da una relazione, quest'ultima deve poter associare una sua istanza a una sola istanza dell'entità, agendo quindi come una funzione[3]. Si dicono **identificatori esterni**.

Bisogna fare attenzione alle modalità di implementazione degli identificatori: in primo luogo non è possibile darne uno a un'istanza di relazione, poiché queste sono già identificate e definite dalla n-upla di istanze delle entità in essa coinvolte; secondariamente

[2]Senza la riga e disegnando due identificatori come nel primo caso, si indica che identificano due cose diverse

[3]In riferimento alla cardinalità, la coppia è $(1, 1)$.

perché l'utilizzo di identificatori implica due ulteriori concetti: **vincoli di definizione**, perché limita la popolazione di un'entità, e **regole di corrispondenza** tra le istanze dell'entità e del sistema informativo. È esattamente per quest'ultima ragione che la scelta degli identificatori va ponderata in base alle proprietà del sistema, poiché aggiungere attributi scollegati dal contesto può portare a una scorretta implementazione della base di dati.

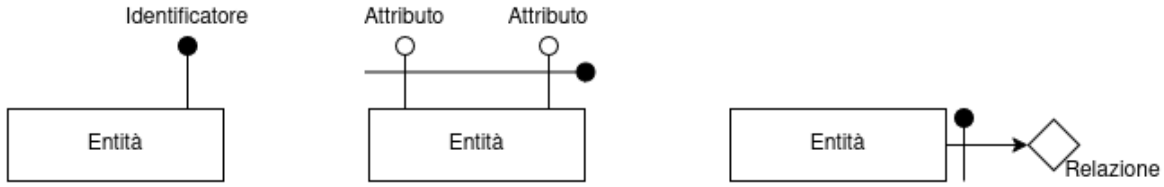


Figure 1.2: Sintassi grafica identificatore

Un ulteriore costrutto utile per definire la totalità delle popolazioni è dato dai **vincoli di cardinalità** sulle relazioni, i quali specificano il numero minimo e massimo di istanze di una relazione R a cui un'istanza di entità E_i deve o può partecipare. Graficamente sono annotati come una coppia (MIN_i, MAX_i), posta sulle linee colleganti entità e relazione; le variabili possono assumere valori diversi:

Valore	MIN_i	MAX_i
0	Partecipazione opzionale alla relazione R	Non assumibile
1	Partecipazione obbligatoria alla relazione R	Massimo una partecipazione di E_i alla relazione R
N	Non assumibile	Nessun limite di partecipazione di E_i alla relazione R
$num > 1$	Esistenza di num istanze di relazione su ogni E_i	Per ogni istanza di entità E_i possono esistere al più num occorrenze della relazione R

Table 1.1: Valori assumibili dalla coppia (min, max)

In tal merito, è possibile associare un vincolo di cardinalità anche ai singoli attributi, annotandolo sulla linea dello stesso. In questo caso, la coppia può assumere i seguenti valori:

- $(1, 1)$: Valori di default, indicano un attributo obbligatorio con massimo una singola istanza.

- $(0, 1)$: Attributo opzionale.
- $(1, N)$: Attributo obbligatorio multivalore.
- $(0, N)$: Attributo opzionale e multivalore.

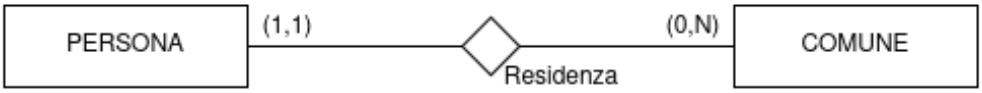


Figure 1.3: Sintassi grafica cardinalità

E ragionando sempre su di essi, è verosimile trovare all'interno di un sistema informativo più attributi con affinità di significato, ovvero con un qualche legame semantico, come per esempio essere una parte di un intero. Per rappresentare questa caratteristica si fa uso degli **attributi composti**, i quali aggregano le suddette proprietà. Graficamente è un costrutto rappresentato da un ovale vuoto, sul quale perimetro emergono le linee degli attributi che lo compongono.

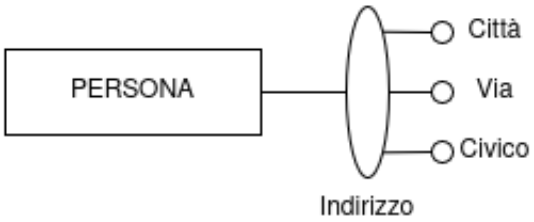


Figure 1.4: Attributo composto

Può inoltre capitare che questo legame concerni anche le entità in toto; in un contesto tale si fa uso della **generalizzazione** delle entità, dove è presente un'entità padre E, più generale e tendenzialmente astratta, e un numero n di entità figlie, le quali erediteranno ogni attributo, identificatore e relazione del padre. Esistono le seguenti modalità di generazione:

- **Totale**: Tutte le istanze del padre sono anche istanze di almeno una figlia.
- **Parziale**: Possono esistere istanze del padre che non sono istanze di almeno una figlia.
- **Esclusiva**: Ogni istanza del padre è anche un'istanza esclusivamente di una figlia.

- **Sovrapposta**: Possono esistere istanze del padre che sono anche istanze di più figlie.

Graficamente la generalizzazione è rappresentata con le entità figlie collegate al padre mediante frecce convergenti, alla cui giunzione è segnalata la modalità applicata mediante le seguenti coppie possibili, le cui iniziali rimandano al tipo di generalizzazione:

$$(t, e) \quad (t, s) \quad (p, e) \quad (p, s)$$

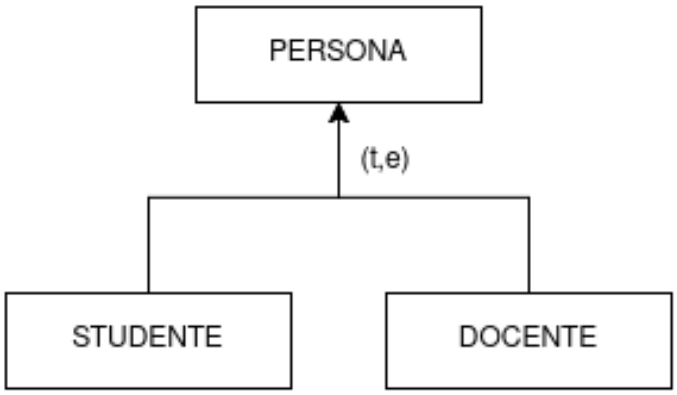


Figure 1.5: Esempio di generalizzazione

Un ulteriore costrutto utile per condensare le scritture è quello della **relazione ternaria**, tuttavia non è usato comunemente, poiché deve rispettare dei vincoli per poter essere utilizzata: richiede sempre la presenza di tre istanze di entità per esistere e non esistono situazioni nelle quali la terna debba essere rappresentata più volte.

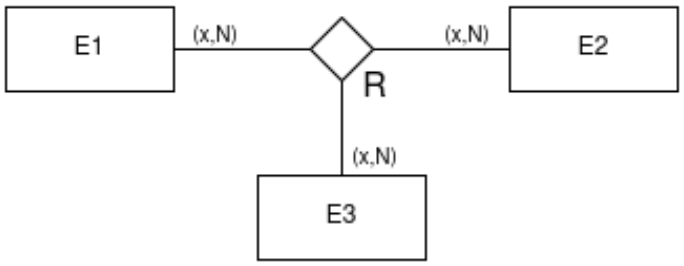


Figure 1.6: Esempio di relazione ternaria

Date queste restrizioni, è comune imbattersi in scenari dove la relazione richieda solo una coppia e non una terna; sarà necessario cambiare la scrittura in una rappresentante tre

relazioni indipendenti fra le entità in gioco. Alternativamente, se la relazione ternaria è l'unione di due relazioni binarie indipendenti, è verosimile che la prima sia ridondante e che possa essere sostituita dalle sole due relazioni indipendenti.

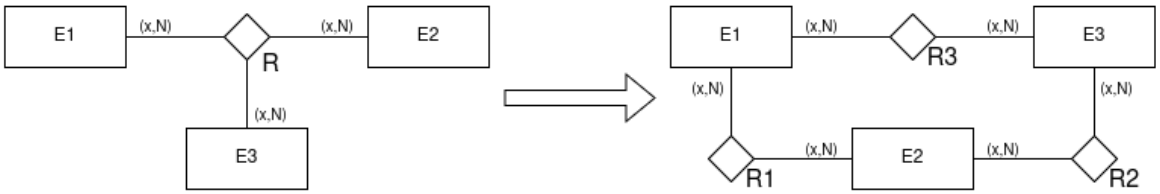


Figure 1.7: Correzione da relazione ternaria a tre indipendenti

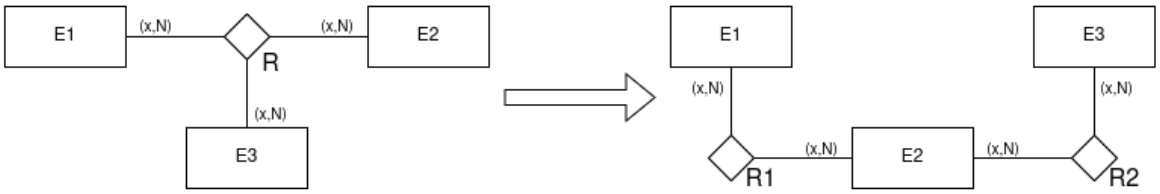


Figure 1.8: Correzione da relazione ternaria a due binarie indipendenti

Tuttavia, le terne possono essere utili per aggiungere uno storico a una relazione binaria; aggiungendo l'entità del tempo sarebbe possibile rappresentare più volte una stessa coppia in tempi diversi. Passando ora alle strategie di approccio alla progettazione concettuale, ne esistono tre principali:

Approccio Top-Down: Considera le specifiche globalmente per produrre uno schema iniziale completo, ma con pochi concetti astratti, per poi raffinarli fino a concretizzarli in ogni dettaglio, ottenendo lo schema finale.

Approccio Bottom-Up: Si decompongono le specifiche in parti elementari per le quali si generano relativi schemi. Il prodotto finito è la fusione di tutti gli schemi precedenti mediante i costrutti del modello ER.

Approccio Inside-Out: Individua alcuni concetti chiave importanti delle specifiche e genera i relativi schemi. Il prodotto finito è la loro fusione mediante i costrutti del modello ER.

Inoltre, è necessario garantire un certo livello di qualità dello schema concettuale, determinato dalla sua analisi, divisa in più step:

1. **Verifica di correttezza**: Uno schema è corretto se utilizza appropriatamente i costrutti del modello concettuale adottato.

2. **Verifica di completezza**: Uno schema è completo se rappresenta tutte le informazioni di interesse e le operazioni indicate nelle specifiche.

3. **Verifica di minimalità**: Uno schema è minimale quando ogni concetto indicato nei requisiti è rappresentato una sola volta. È quindi privo di ridondanze.

4. **Verifica di leggibilità**: Uno schema è chiaro quando si ha cura della rappresentazione grafica. È una caratteristica curabile con una scelta di nomi appropriata ed evitando di disegnare linee sovrapposte o contorte.

1.4 Progettazione logica

1.5 Esercizi svolti

Chapter 2

Interazione con un database

Chapter 3

RDBMS e SQL in postgreSQL

Chapter 4

Query e transazioni

Chapter 5

Accesso a database da programmi Python

Chapter 6

Uso di DBMS, NoSQL e MongoDB

Chapter 7

Database Management Systems

Chapter 8

Interrogazioni in SQL

Chapter 9

Architettura NoSQL MongoDB